

Paper Review: Slicing Aided Hyper Inference (SAHI)

Akyon et al., 2022

Maryam Amjad

SmartSites Technical Task

Part A

What is SAHI and why does standard detection fail on small objects

Standard detectors like faster R-CNN and yolo were trained on datasets like MS COCO and ImageNet which are low resolution but objects in them are very large, covering around 60% of the image height. So when we deploy these models on real world footage like CCTV where workers appear as a very tiny speck, the model has never seen objects that small during training. It does not detect them.

One obvious fix would be to increase the size of the image so the objects become larger and detectable, but this approach demands significantly higher gpus and memory resources. According to DORI criteria (the surveillance industry standard), a minimum of 108 pixels in height is required just to recognize an object in full HD footage. Most distant workers fall far below this threshold.

SAHI solves this differently. Instead of scaling the full image, it slices the image into overlapping patches, runs the detector on each patch separately, and then merges all the detections back into the original image coordinates using NMS. Each patch is small enough to fit in memory but since the object now occupies more of the patch, the detector can actually see it. The key advantage is that SAHI is generic, it works on top of any existing detector without any retraining.

Tradeoffs: Speed, False Positives, and Overlap Handling

Speed

SAHI increases computation with the number of patches. It runs the detector once per patch instead of once per image. For real time CCTV this is a real cost. However memory stays fixed since only one patch is processed at a time.

False Positives

Slicing introduces a new source of false positives specifically for large objects. A large object that spans multiple patches can get detected partially in several patches, and when those detections are merged back, they can create duplicate or misaligned boxes for that large object.

Overlap Handling

The paper uses 25% overlap between patches so no object gets cut off at the edges. After detection, all boxes from all patches are merged back into the original image. Since the same object was seen in multiple patches, there will be duplicate boxes. NMS removes duplicates by

keeping only the most confident box. The downside is in crowded scenes it can accidentally delete a real detection thinking it is a duplicate.

Challenges applying SAHI to construction site CCTV

Construction site CCTV is much harder than the drone datasets the paper tested on. The following challenges are expected:

- Camera distance: Fixed cameras are mounted high up so workers far from the camera appear extremely small, sometimes just 10-15 pixels tall. SAHI helps but patch size needs to be tuned per camera.
- Camera and lens quality: The resolution and lens quality of CCTV cameras varies a lot. A cheap low resolution camera with a poor lens introduces distortion, softness, and chromatic aberration on top of the compression artifacts. So the model should be robust enough to handle such situations i.e. the differentiation of lens should not matter.
- Compressed streams: CCTV footage is heavily compressed which introduces blur and artifacts. Small objects lose even more detail and the detector picks up noise instead of actual features.
- Occlusion: Workers constantly go behind cranes or vehicles. A partially visible worker may only show up in one patch as a partial body which makes detection unreliable.
- Lighting changes: Construction sites have daylight, artificial lighting and night conditions. A model working well in the afternoon can completely miss workers at night or under glare. SAHI does not solve this.
- Crowded scenes: Workers standing close together have overlapping bounding boxes. NMS can merge two separate workers into one detection causing undercounting.
- Motion blur: Workers moving fast in low light produce blurry frames. A blurry worker at small pixel size becomes nearly impossible to detect even with slicing.

Designing a live construction site detection pipeline, beyond SAHI

SAHI alone is just an inference improvement. For a real production pipeline on a live construction feed, these additions would make it actually practical:

- Kalman filter based tracking on top of SAHI: SAHI only gives detections per frame, it has no idea who is who across frames. Adding any Kalman filter based tracker like SORT or ByteTrack after SAHI keeps the same worker ID across frames. This matters for productivity tracking because you need to know worker A was idle for 10 minutes, not just that someone appeared in frame 1 and frame 1000.
- Frame skipping with tracking: At 30fps you do not need to run inference on every single frame. Running inference every 3rd frame and using the tracker to propagate identities across the skipped frames gives almost the same result at a third of the compute cost.
- Use detection count to adjust slice size: If a slice keeps returning zero detections it is probably covering empty background, so increase its size. If a slice keeps returning too

many crowded detections, decrease the size so workers can be separated. Simple feedback loop, no ML needed.

- Resize patches to match yolo input size: yolo was trained on 640x640 square images. If patches after slicing are not square, the model is seeing something different from what it was trained on. Resizing each patch to exactly 640x640 before inference is a small fix that helps accuracy.
- In the beyond SAHI section you can add this:
- Replace YOLO backbone with a foundation model: YOLO uses a standard CNN backbone that was trained on a limited dataset. Replacing it with a self supervised foundation model like DINOv3 as the feature extractor gives much richer visual representations since DINOv3 was trained on massive data without any labels. The detection head sits on top of these features as a simple linear probe. This means even small distant workers with very few pixels carry more meaningful features going into the detector, so SAHI and the detector together perform better than either would with a standard backbone.